SPM Modular Project

Module 2:Partitioned Hash Join with Duplicates

Student: Karol Jinet Cardona Bolanos

Mat. 702023

Academic Year 2025/2026

1 Design Choices

Partition Mapping Function: The placeholder `compute_partition_id` provided by the professor is replaced with `multiply_shift` from Module 1 (Dietzfelbinger universal hashing):

$$\texttt{pid} = (\texttt{key} \times \texttt{MULT_PRIME}) \gg (64 - \log_2 P)$$

where $\texttt{MULT_PRIME} = \lfloor 2^{64}/\varphi \rfloor = \texttt{0x9E3779B97F4A7C15}$. This function was selected among the five candidates evaluated in Module 1 based on the best trade-off between throughput and distribution quality. While `mask` is faster (1182 Mkeys/s), it maps only low-order bits, creating skewed partitions and thread load imbalance. Conversely, `murmur_fmix64` avoids skew but is too slow (538 Mkeys/s). `multiply_shift` strikes the ideal balance, achieving 1111 Mkeys/s in scalar mode, with a guaranteed uniform distribution (verified in Module 1). The same function is used consistently in the sequential baseline and all three parallel versions.

Parameter Selection: P=256 is chosen as the number of partitions. Benchmarks in Module 1 confirm that P does not affect execution time, so P=256 provides near-perfect distribution (CV ≈ 0.002) at no performance cost. The benchmark uses NR= 50M and NS= 80M records with max-key= 1,000,000, producing approximately 50 duplicate matches per key on average. NR and NS are intentionally different to reflect a realistic asymmetric join scenario, where the build relation R is smaller than the probe relation S.

Build-phase Hash Table: The reference implementation uses `std::unordered_map` for the local build phase. This structure relies on heap-allocated bucket chains: each lookup follows scattered pointers in memory, producing random cache misses that are particularly costly when the map contains many distinct keys. It is replaced in all four versions (sequential and parallel) with `FlatCountMap`, a flat open-addressing hash table with linear probing:

$$\texttt{idx} = (\texttt{key} \times \texttt{MULT_PRIME}) \;\&\; \texttt{mask}$$

All slots are stored in a single contiguous `std::vector<Slot>`, with the table size set to the next power of two $\geq 2 \times$ capacity (load factor ≤ 0.5). Empty slots are marked with the sentinel `EMPTY_KEY = ~0ULL`. This choice eliminates pointer indirection and maximises spatial locality: consecutive probes walk a contiguous array rather than chasing heap pointers. Since each partition is processed by exactly one thread, `FlatCountMap` does not alter the parallel structure of the algorithm — it reduces absolute latency uniformly across all thread counts.

Pipeline Phases: The algorithm is structured as four sequential phases applied to both relations. The key property exploited for parallelisation is that after Phase 3, every partition is a self-contained, independent segment of memory: Phases 1, 3, and 4 are parallelised incrementally across three versions (Section 2).

Table 1: Pipeline phases and their computational characteristics

Phase	Description	Cost	Notes
1. Histogram	Count records per partition	$O(N)$	Embarrassingly parallel
2. Prefix sum	Compute start offsets	$O(P)$	P=256: negligible
3. Scatter	Reorder into contiguous segments	$O(N)$	Main data-movement phase
4. Local join	Build on R_p, probe with S_p	$O(N/P)$	Fully independent per partition

2 Parallelisation Strategy

Three parallel versions were developed *incrementally* to isolate the performance contribution of each phase. All versions use `std::thread` with no atomics or mutexes.

v1 — Parallel Local Join Only

The partitioning pipeline (histogram, prefix sum, scatter) is left sequential, identical to the professor's reference. Only the loop over the P=256 partitions is parallelised: thread t processes partitions t, $t{+}T$, $t{+}2T, \ldots$ (stride-T round-robin). Each thread accumulates results in its own private `JoinResult`, so no synchronisation is needed. The main thread sums the per-thread results after all threads complete.

This version exposes an important bottleneck: the sequential partitioning phase takes $\sim$2.47 s regardless of T, which caps the achievable speedup.

v2 — Parallel Join + Parallel Histogram

Each thread independently counts records in its own contiguous chunk of the input. Since each thread writes only to its private histogram row, no write conflicts arise. After all threads complete, the main thread merges the local histograms by summing element-wise. The scatter remains sequential in this version.

v3 — Fully Parallel (Final Version)

The scatter is parallelised using a *conflict-free write scheme*. Before scattering, the main thread computes `thread_cursor`$[t]$[`pid`], the exact starting write position reserved for thread t within partition `pid`:

$$\texttt{thread_cursor}[t][\texttt{pid}] = \texttt{global_begin}[\texttt{pid}] + \sum_{k=0}^{t-1} \texttt{local_hist}[k][\texttt{pid}]$$

Since each thread writes only to its pre-assigned range, no two threads ever access the same output position, eliminating the need for atomics entirely.

Additionally, v3 uses `alignas(64)` on the per-thread result accumulators (`PaddedJoinResult`) to prevent *false sharing*. Without padding, each `JoinResult` occupies only 24 bytes (three `uint64_t` fields), so multiple accumulators share the same 64-byte cache line. When two threads write to different accumulators on the same cache line, the hardware incorrectly treats it as a conflict and forces a cache invalidation, even though no data is actually shared. Aligning each accumulator to a full cache line boundary eliminates this effect.

Note on prefix sum: The prefix sum is left sequential in all three versions. It operates on only P=256 integers, a workload so small that its execution time is negligible compared to the data phases. Parallelising it would require spawning threads and a barrier synchronisation whose overhead would far exceed any potential gain, a clear case where the cost of coordination outweighs the benefit of parallelism.

3 Correctness Verification

Two complementary strategies are used. For small inputs (`NR`, `NS` ≤ 500), the `make check` target runs a brute-force $O(|R|\times|S|)$ verifier and compares `join_count`, `checksum1`, and `checksum2` between the sequential baseline and all three parallel versions, all of which pass. For large inputs, correctness is verified by comparing the same three output values between the sequential and parallel runs: Table 2 confirms bit-exact equivalence across all versions at `NR`=50M, `NS`=80M, T=8.

Table 2: Correctness verification (NR=50M, NS=80M, P=256)

Version	join_count	checksum1	checksum2
seq	4 000 007 100	11403703363617461081	11564485458395885908
v1 T=8	4 000 007 100	11403703363617461081	11564485458395885908
v2 T=8	4 000 007 100	11403703363617461081	11564485458395885908
v3 T=8	4 000 007 100	11403703363617461081	11564485458395885908

4 Performance Results

Benchmarks were executed on `spmcluster` (`--partition=normal`), with NUMA pinning via `numactl --cpunodebind=0 --membind=0` to improve measurement reproducibility. Input parameters: NR=50M, NS=80M, P=256, seed= 13, max-key= 1,000,000.

4.1 Strong Scaling

Table 3: Strong scaling — wall-clock time (s) and speedup vs. sequential baseline (3.49 s)

T	v1 (s)	Sp.	v2 (s)	Sp.	v3 (s)	Sp.
1	3.45	1.01×	3.45	1.01×	3.45	1.01×
2	3.06	1.14×	2.98	1.17×	2.03	1.72×
4	2.80	1.25×	2.68	1.30×	1.23	2.85×
8	2.69	1.30×	2.54	1.38×	0.84	4.15×
16	2.69	1.30×	2.55	1.37×	0.71	**4.91**×

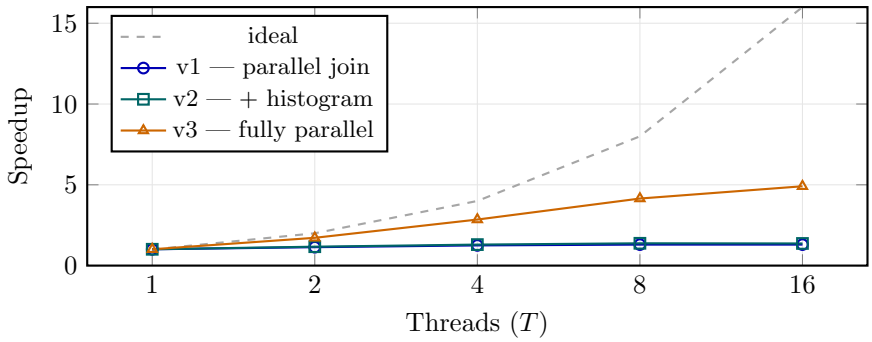


Figure 1: Strong scaling speedup vs. sequential baseline (3.49 s). Dashed line = ideal linear speedup.

v1 and v2 show limited speedup: even at T=16, they reach only 1.30× and 1.37× respectively. The sequential partitioning phase takes ∼2.47 s regardless of T, representing ≈ 71% of the total single-thread execution time and acting as a fixed floor on the total execution time.

v3 scales significantly better, reaching **4.91**× at T=16. The gain between T=8 (4.15×) and T=16 (4.91×) is moderate, suggesting the system is approaching a memory bandwidth limit.

4.2 Phase Breakdown

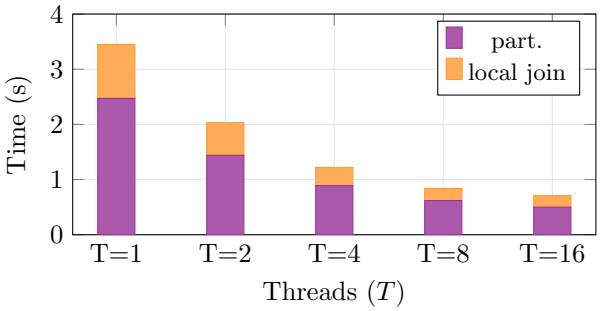
Table 4 and Figure 2 show the per-phase timing for v3, separating partitioning (histogram + scatter for both R and S) from the local join. At T=1, partitioning dominates (2.47 s vs 0.98 s for the join), reflecting the reduced join cost from `FlatCountMap`. As T increases, both phases scale well: partitioning drops from 2.47 s to

0.50 s (4.94×), and join from 0.98 s to 0.21 s (4.67×). At T=16 the two phases converge (0.50 s vs. 0.21 s), with partitioning remaining the slightly larger term, as it involves two full scatter passes over the data.

Table 4: v3 phase breakdown (NR=50M, NS=80M, P=256)

T	Part. (s)	Join (s)	Total (s)
1	2.47	0.98	3.45
2	1.44	0.59	2.03
4	0.89	0.33	1.23
8	0.62	0.22	0.84
16	0.50	0.21	0.71

Figure 2: v3 phase breakdown stacked.



4.3 Weak Scaling

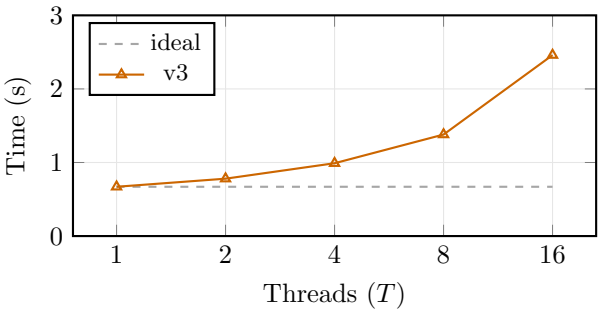
Weak scaling fixes the work per thread: NR starts at 10M and NS at 16M for T=1, both scaling proportionally with T. Ideal behaviour would produce constant time and efficiency of 1.00.

Efficiency remains acceptable up to T=4 (0.68), with a gradual degradation caused by the growing synchronisation overhead of the parallel histogram and scatter phases as both the thread count and dataset size increase simultaneously. The drop at T=8 (efficiency 0.49) and the sharp drop at T=16 (efficiency 0.27, time 2.46 s) are consistent with memory bandwidth saturation: at T=16 the dataset reaches 416M total records (∼3.3 GB), fully saturating the DRAM bandwidth of the node. This is consistent with the strong scaling results, where the same saturation effect was observed between T=8 and T=16.

Table 5: Weak scaling — v3

T	NR	NS	Time (s)	Eff.
1	10M	16M	0.67	1.00
2	20M	32M	0.78	0.87
4	40M	64M	0.99	0.68
8	80M	128M	1.38	0.49
16	160M	256M	2.46	0.27

Figure 3: Weak scaling execution time.



4.4 Impact of Key Distribution

To evaluate the sensitivity of the algorithm to key distribution, we vary max-key while keeping all other parameters fixed (NR=50M, NS=80M, P=256, T=8).

Table 6: Impact of max-key on seq and v3 performance (T=8)

max-key	dup./key	join seq (s)	join par (s)	total seq (s)	total par (s)
100	∼500K	0.91	0.17	3.23	0.76
10K	∼5K	0.82	0.15	3.29	0.77
100K	∼500	0.84	0.15	3.31	0.77
1M	∼50	1.02	0.21	3.49	0.83
10M	∼5	1.28	0.38	3.77	1.01
50M	∼1	2.18	0.62	4.65	1.24

Two observations emerge from Table 6. First, the partitioning time remains completely insensitive to max-key (∼2.47 s sequential, ∼0.62 s parallel), confirming that it depends only on N and not on key distribution. Second, the join time grows with max-key: from 0.82 s (max-key= 10K) to 2.18 s (max-key= 50M) in the sequential case. This is explained by cache pressure: with fewer duplicates per key, each partition's `FlatCountMap` contains more distinct entries. At low max-key values the maps are tiny and fit entirely in L1/L2 cache, so lookups are fast; at high max-key values (∼195K distinct entries per partition at max-key= 50M with P=256) the table no longer fits in cache and random-access latency dominates. The parallel version shows the same trend but is less affected, as each thread works on a smaller independent subset of partitions.

5 Discussion

Incremental parallelisation analysis: The three-version approach directly exposes the cost of each sequential bottleneck.

In v1, only the local join is parallelised. The partitioning phase (histogram, prefix sum, scatter) remains entirely sequential, taking ∼2.47 s regardless of T, representing $\approx 71\%$ of the single-thread execution time. By Amdahl's Law, the theoretical maximum speedup is $1/f_{\text{seq}} \approx 1.41\times$, consistent with the observed plateau at 1.30×.

In v2, the histogram is also parallelised. The improvement is again marginal — only ∼0.13 s at T=16 (from 2.47 s to 2.34 s) — because the scatter phase dominates partitioning time and remains sequential. As a result, v2 achieves only a modest improvement over v1.

In v3, the scatter is parallelised as well. This removes the dominant bottleneck: the partitioning phase now scales with T, dropping from 2.47 s to 0.50 s at T=16 (Table 4), and the peak speedup reaches **4.91**×.

Memory bandwidth as the limiting factor: Strong scaling of v3 plateaus between T=8 (4.15×) and T=16 (4.91×), a gain of less than 0.8× when doubling the thread count. This behaviour is consistent with memory bandwidth saturation: the workload is dominated by sequential reads and writes over large arrays, and beyond T=8 additional threads compete for the same memory bus without reducing the data movement cost.

Weak scaling degradation: As discussed in Section 4, efficiency degrades from 0.87 (T=2) to 0.27 (T=16), with a sharp drop at T=16 consistent with the memory bandwidth saturation observed in strong scaling.

Comparing strong and weak scaling at T=16: in strong scaling the time drops to 0.71 s on a fixed dataset, while in weak scaling it rises to 2.46 s on a 16× larger dataset. This confirms that the implementation scales well when the dataset fits comfortably in the memory hierarchy, but degrades when data movement dominates.

6 Conclusions

The parallel partitioned hash join achieves a peak speedup of **4.91**× over the sequential baseline with 16 threads, reducing wall-clock time from 3.49 s to 0.71 s on 130M total records (`NR`=50M, `NS`=80M).

The incremental three-version approach provides clear experimental evidence for each design decision:

- Parallelising only the join phase (v1) is insufficient: the scatter must also be parallelised to remove the dominant sequential bottleneck.
- Adding parallel histogram (v2) yields only marginal improvement because the scatter dominates partitioning time and remains sequential.
- Parallelising the scatter (v3) removes the bottleneck entirely, allowing both phases to scale with T and achieving 4.91× at T=16.

The system is ultimately memory-bound: strong scaling plateaus between T=8 and T=16, and weak scaling efficiency drops to 0.27 at T=16 as the dataset saturates the DRAM bandwidth of the node.